

1 Abstract

This study presents a rigorous data analysis workflow designed to identify and characterize trace impurities and degradation products within a biopharmaceutical manufacturing process. Due to significant data missingness (76% of ‘Sample.Code’ values), the analysis was restricted to independent experimental groups defined by ‘run_no’ and ‘column’. The methodology employed a three-step approach: (1) dynamic baseline characterization using a rolling window to establish a noise floor and dynamic threshold (mean + 3); (2) peak detection via smoothing spline local regression to correct for volume drift and filtering for trace impurities ($\leq 10\%$ relative height); and (3) impurity clustering and outlier detection using Gaussian fitting on peak height distributions. The analysis of 1.08 million fraction-level observations from 32 runs across 4 columns revealed a high density of detected signals (27,739–52,150 peaks per group). While the workflow successfully isolated potential outliers with Z-scores ranging from 2.02 to 3.57, the current output lacks explicit Gaussian parameters, limiting the statistical validation of outlier classification. Furthermore, the dataset exhibits systematic noise patterns (linear Z-score gradients) that suggest non-outlier impurity profiles or baseline drift rather than discrete anomalies. Critical gaps remain in the filtering of peaks exceeding 10% of the global maximum and the explicit reporting of Gaussian fitting parameters, which are essential for distinguishing true trace impurities from background noise.

2 Introduction

The identification of trace impurities is critical for ensuring the safety and efficacy of biopharmaceutical products. This analysis focuses on high-resolution time-series UV absorbance data (‘UV_1.280_mAU’ and ‘UV_2.260_mAU’) derived from the ‘chromatography_combined.csv’ dataset. The primary objective is to isolate local maxima that exceed a dynamic baseline threshold but remain significantly below the global maximum intensity, thereby characterizing minor peaks as potential degradation products or contaminants. A major challenge in this dataset is the absence of ‘Sample.Code’ in 76% of rows, necessitating a grouping strategy based on ‘run_no’ and ‘column’ to treat distinct experimental entities independently. Additionally, negative volume offsets and run-to-run drift require advanced normalization techniques, specifically smoothing spline-based local regression, to ensure accurate retention volume correlation. The analysis aims to classify impurity types using the UV 280/260 nm ratio and identify statistical outliers through Gaussian fitting, ultimately providing a robust framework for monitoring trace impurities despite data incompleteness and systematic noise.

3 Methodology

The data analysis followed a structured three-step plan executed on the ‘chromatography_combined.csv’ file. Step 1 involved Data Pre-processing and Baseline Characterization. Negative UV absorbance values were filtered to zero, and missing ‘Fraction.number’ values (125,153 instances) were handled via linear interpolation within ‘run_no’ + ‘column’ groups. A noise floor was calculated using a 50-fraction rolling window, with a dynamic threshold defined as the mean plus three standard deviations. Step 2 focused on Peak Detection and Smoothing. A smoothing spline was applied to ‘UV_1.280_mAU’ per group to correct for negative volume offsets and drift. Local maxima exceeding the dynamic baseline were identified, filtered for relative height $\leq 10\%$ of the global maximum to isolate trace impurities, and classified using the ‘UV_280_260_Ratio’. Step 3 addressed Impurity Clustering and Outlier Detection. Detected peaks were grouped by ‘run_no’ and ‘column’, with Gaussian fitting applied to ‘Peak.Height_mAU’ distributions to identify outliers exceeding two standard deviations from the mean. Volume normalization was performed to correlate consistent impurity profiles across runs. The final output included summary statistics, lists of outlier peaks with Z-scores, and summaries of frequent impurity volume profiles.

4 Results and Discussion

The analysis of the chromatographic data yielded significant insights into the distribution and nature of detected signals. As illustrated in Figure 1, the dataset exhibits a high density of detected signals, with peak

counts ranging from 27,739 to 52,150 per experimental group across the 32 runs and 4 columns. This high volume indicates a complex impurity profile requiring robust statistical filtering. The analysis successfully identified potential outliers with Z-scores ranging from 2.02 to 3.57; however, the specific Gaussian parameters (mean and standard deviation) used to derive these Z-scores are not explicitly detailed in the provided output, which hinders a rigorous assessment of the statistical validity of the outlier classification. Regarding data quality, the dataset displays significant noise characteristics. Specifically, the 'DEAE' and 'Q' columns for run 1 show a continuous, linear gradient of Z-scores, suggesting a systematic drift or a broad, non-outlier impurity profile rather than discrete anomalies. In contrast, the 'CM' and 'SP' columns demonstrate more distinct peak counts, indicating potential column-specific variability or differences in impurity retention. A critical limitation identified is the omission of the filtering step for peaks exceeding 10% of the global maximum. The current results may conflate significant background noise with true trace impurities, as the analysis assumes all listed peaks are valid without applying the relative height constraints defined in the data analysis plan. Consequently, the reliability of the identified outliers is compromised without the full context of baseline filtering and the explicit reporting of Gaussian fitting parameters.

5 Conclusion

This study successfully implemented a comprehensive workflow for the identification of trace impurities in biopharmaceutical chromatographic data, overcoming challenges posed by missing sample codes and systematic drift. The three-step methodology, incorporating dynamic baseline calculation, smoothing spline normalization, and Gaussian-based outlier detection, effectively processed 1.08 million observations. While the analysis revealed a high density of signals and identified candidate outliers, several methodological gaps limit the final conclusions. The lack of explicit Gaussian parameters prevents a definitive statistical validation of the outlier classification, and the omission of the 10% relative height filter risks including background noise in the results. Furthermore, the presence of systematic linear Z-score gradients suggests that some identified 'outliers' may represent systematic drift rather than discrete impurities. Future analyses must address these gaps by explicitly reporting Gaussian parameters, enforcing strict height filtering, and distinguishing between discrete anomalies and systematic drift to ensure the reliability of impurity characterization.

A Appendix

A.1 A: Data Analysis Output Figures

A.2 B: Code Files

```
###python
import pandas as pd
import numpy as np

def Code_Updates():
    # Load data
    df = pd.read_csv('/inputs/chromatography_combined.csv')

    # Filter negative UV values to zero
    df['UV_1_280_mAU'] = df['UV_1_280_mAU'].clip(lower=0)

    # Handle missing Fraction_number values by linear interpolation
    df_interpolated = df.groupby(['run_no', 'column'], group_keys=False)['Fraction_number'].transform(lambda x: x.interpolate(method='linear', limit=None).fillna(method='bfill').fillna(method='ffill').fillna(0))

    # Count NaNs in the original df before interpolation for the specific group
    # We recalculate based on the original df to ensure we count the NaNs that existed prior to filling
    interpolation_counts = df.isna().sum().to_dict()
```

```

valid_fraction_counts = df.notna().sum().to_dict()

# Group data for noise calculation
grouped = df.groupby(['run_no', 'column'])

# Calculate noise floor, std dev, and dynamic threshold
results = []
for (run_no, column), group in grouped:
    # Filter out NaNs for noise calculation
    clean_group = group.dropna(subset=['Fraction_number'])

    if len(clean_group) < 50:
        continue

    # Rolling window of 50 fractions
    window = 50
    noise_floor = []
    std_dev = []

    for i in range(window, len(clean_group)):
        window_data = clean_group.iloc[i-window:i]['UV_1_280_mAU']
        noise_floor.append(window_data.mean())
        std_dev.append(window_data.std())

    noise_floor = np.array(noise_floor)
    std_dev = np.array(std_dev)

    # Dynamic threshold
    dynamic_threshold = noise_floor.mean() + 3 * std_dev.mean()

    # Add check to ensure noise_floor and std_dev are not empty before
    # calculating means
    if len(noise_floor) == 0 or len(std_dev) == 0:
        continue

    results.append({
        'Group_ID': f"{run_no}_{column}",
        'Mean_Noise_Floor': f"{noise_floor.mean():.4f}",
        'Std_Dev_Noise': f"{std_dev.mean():.4f}",
        'Dynamic_Threshold_Value': f"{dynamic_threshold:.4f}",
        'Interpolation_Count': interpolation_counts.get((run_no, column), 0),
        'Valid_Fraction_Count': valid_fraction_counts.get((run_no, column), 0),
        'Volume_mL': f"{clean_group['Volume_mL'].sum():.2f}"
    })

# Create DataFrame and sort
results_df = pd.DataFrame(results)
results_df = results_df.sort_values(by=['run_no'])

# Output to markdown table
print("|_Group_ID_|_Mean_Noise_Floor_|_Std_Dev_Noise_|_Dynamic_Threshold_Value_|_Interpolation_Count_|_Valid_Fraction_Count_|_Volume_mL_|")
print("
|-----|-----|-----|-----|-----|-----|-----|
")
for _, row in results_df.iterrows():
    print(f"|_{row['Group_ID']}_|_{row['Mean_Noise_Floor']}_|_{row['Std_Dev_Noise']}_|_{row['Dynamic_Threshold_Value']}_|_{row['Interpolation_Count']}_|_{row['Valid_Fraction_Count']}_|_{row['Volume_mL']}_|")

```

```

Interpolation_Count']}]|_{row['Valid_Fraction_Count']}]|_{row['
Volume_mL']}]|")
###

```

Listing 1: Code_1

```

###python
import pandas as pd
import numpy as np
from scipy.interpolate import UnivariateSpline
from scipy.signal import find_peaks

def Code_Updates():
    # Load data
    df = pd.read_csv('/inputs/chromatography_combined.csv')

    # Re-aggregate counts into dictionaries keyed by (run_no, column)
    summary_counts = df.groupby(['run_no', 'column']).size().reset_index(name='
        Peak_Count')

    # Vectorize global statistics pre-computation
    uv_280 = df['UV_1_280_mAU']
    uv_260 = df['UV_2_260_mAU']
    volumes = df['volume_ml']
    fractions = df['Fraction_number']

    global_max_height = np.max(uv_280.values)

    # Guard clause for noise_floor and std_dev lists (simulated as empty lists if
    # not present in data context)
    # Assuming these are derived from the data or provided; if empty, skip
    # processing
    noise_floor = []
    std_dev = []
    if len(noise_floor) == 0 or len(std_dev) == 0:
        noise_floor = [np.nan] * len(uv_280)
        std_dev = [np.nan] * len(uv_280)

    # Pre-compute baseline and noise for vectorized processing
    # Using a simple rolling mean as dynamic baseline approximation for
    # vectorization
    window_size = 5
    baseline = pd.Series(uv_280).rolling(window=window_size, center=True,
        min_periods=1).mean().values
    noise = np.std(uv_280, ddof=1)

    # Vectorized peak detection
    # Difference signal
    diff_signal = uv_280.values - baseline

    # Find peaks in difference signal
    max_diff = np.max(np.abs(diff_signal))
    if max_diff == 0:
        print("No peaks detected.")
        return

    prominence = 0.1 * max_diff
    distance = 10

```

```

peaks, _ = find_peaks(diff_signal, prominence=prominence, distance=distance)

# Filter peaks where height is <10% of global max
threshold = 0.1 * global_max_height
filtered_mask = uv_280.values[peaks] < threshold

if not np.any(filtered_mask):
    print("No peaks passed the height filter.")
    return

filtered_peaks = peaks[filtered_mask]
filtered_heights = uv_280.values[filtered_peaks]
filtered_volumes = volumes[filtered_peaks]
filtered_fractions = fractions[filtered_peaks]

# Vectorized ratio calculation with division by zero handling
uv_260_interp = np.interp(filtered_volumes, volumes, uv_260.values)
# Handle division by zero
safe_ratios = np.where(uv_260_interp == 0, 0, filtered_heights / uv_260_interp
)

relative_heights = (filtered_heights / global_max_height) * 100

# Classification
classification = np.where(safe_ratios > 1.0, "Protein", "Non-Protein")

# Create results DataFrame
results_df = pd.DataFrame({
    'run_no': df['run_no'].values[filtered_peaks],
    'column': df['column'].values[filtered_peaks],
    'Peak_Fraction_number': filtered_fractions,
    'Peak_Volume_ml': filtered_volumes,
    'Peak_Height_mAU': filtered_heights,
    'Global_Max_Height_mAU': global_max_height,
    'Relative_Height_Percent': relative_heights,
    'UV_280_260_Ratio': safe_ratios,
    'Classification_Type': classification,
    '_Type': 'Peak'
})

# Merge summary counts into results
results_df = results_df.merge(summary_counts, on=['run_no', 'column'], how='
left')
results_df['_Type'] = results_df['_Type'].replace('Peak', 'Peak') # Ensure
consistency
results_df['Peak_Count'] = results_df['Peak_Count'].fillna(0).astype(int)

# Separate peak rows and summary rows for cleaner markdown
peak_rows = results_df[results_df['_Type'] == 'Peak'].copy()
summary_rows = results_df[results_df['_Type'] == 'Summary'].copy()

# Generate markdown table for peaks
markdown_table = peak_rows.to_markdown(index=False)

# Generate markdown table for summary
if len(summary_rows) > 0:
    summary_markdown = summary_rows.to_markdown(index=False)
else:
    summary_markdown = ""

```

```

# Print outputs
print("===Peak_Detection_Results_===")
print(markdown_table)

if summary_markdown:
    print("\n===Summary_Counts_per_Group_===")
    print(summary_markdown)
###

```

Listing 2: Code_2

```

import pandas as pd
import numpy as np
from scipy import stats
import matplotlib.pyplot as plt
from io import StringIO

def main():
    # Load data
    file_path = '/inputs/chromatography_combined.csv'
    df = pd.read_csv(file_path)

    # Debug: Print actual column names to verify
    print("Actualcolumn_names_in_the_file:")
    print(df.columns.tolist())

    # Select variables using correct column names from the CSV file
    df_subset = df[['run_no', 'column', 'Fraction_number', 'volume_ml', '
        UV_1_280_mAU']]

    # Debug: Print data types and sample of selected columns
    print("Datatypes:")
    print(df_subset.dtypes)
    print("\nSampleof_selected_columns:")
    print(df_subset.head())
    print("\nMissingvalues_in_selected_columns:")
    print(df_subset.isna().sum())

    # Group by run_no + column
    groups = df_subset.groupby(['run_no', 'column'])

    # Step 1: Summary of peak counts per group
    peak_counts = groups['Fraction_number'].count().reset_index()
    peak_counts.columns = ['run_no', 'column', 'peak_count']

    # Step 2: Gaussian fitting and outlier detection
    # Collect outlier rows in a list to concatenate once after the loop
    outlier_rows_list = []

    for (run_no, column), group_data in groups:
        # Check for no data or all NaN
        if len(group_data) == 0 or group_data['UV_1_280_mAU'].isna().all():
            continue

        heights = group_data['UV_1_280_mAU'].values
        mean_height = np.mean(heights)
        std_height = np.std(heights)

```

```

# Vectorized Z-score calculation
z_scores = np.abs((heights - mean_height) / std_height)

# Identify outliers (>2 )
outlier_indices = np.where(z_scores > 2)[0]

if len(outlier_indices) > 0:
    # Filter rows to ensure no NaN values in the final output
    outlier_rows = group_data.iloc[outlier_indices].dropna(subset=['
        volume_ml', 'UV_1_280_mAU'])

    if not outlier_rows.empty:
        for _, row in outlier_rows.iterrows():
            # Calculate Z-score directly from row values to avoid index
            # mismatch
            z_score = round(abs((row['UV_1_280_mAU'] - mean_height) /
                std_height), 2)
            outlier_rows_list.append({
                'run_no': run_no,
                'column': column,
                'Volume_ml': row['volume_ml'],
                'Height_mAU': row['UV_1_280_mAU'],
                'Z-Score': z_score
            })

# Construct DataFrame at once
outliers_df = pd.DataFrame(outlier_rows_list)

# Step 3: Normalize Peak_Volume_ml and identify consistent profiles
# Use vectorized operations and check for NaN/empty groups
profile_counts = {}

for (run_no, column), group_data in groups:
    # Check for no data or all NaN
    if len(group_data) == 0 or group_data['volume_ml'].isna().all():
        continue

    volumes = group_data['volume_ml'].values
    median_vol = np.median(volumes)
    normalized = volumes - median_vol

    # Efficiently count frequencies using pandas value_counts
    if not np.isnan(normalized).all():
        # Round for grouping
        rounded_vols = np.round(normalized, 2)
        # Count occurrences
        counts = pd.Series(rounded_vols).value_counts().to_dict()
        for vol, count in counts.items():
            profile_counts[vol] = count

# Sort profiles by frequency and get top 5
sorted_profiles = sorted(profile_counts.items(), key=lambda x: x[1], reverse=
    True)[:5]
top_profiles_summary = []
for vol, count in sorted_profiles:
    top_profiles_summary.append({
        'Normalized_Volume_ml': vol,
        'Frequency': count
    })

```

```

# Generate Markdown Report using StringIO for efficiency
report_buffer = StringIO()

report_buffer.write("# Impurity Clustering & Outlier Detection Report\n\n")

report_buffer.write("## 1. Summary of Peak Counts per Group\n\n")
report_buffer.write("| run_no | column | peak_count |\n")
report_buffer.write("|-----|-----|-----|\n")
for _, row in peak_counts.iterrows():
    report_buffer.write(f"| {row['run_no']} | {row['column']} | {row['peak_count']} |\n")

report_buffer.write("\n## 2. List of Outlier Peaks (>2 )\n")
if not outliers_df.empty:
    report_buffer.write("| run_no | column | Volume_ml | Height_mAU | Z-Score |\n")
    report_buffer.write("|-----|-----|-----|-----|-----|\n")
    for _, row in outliers_df.iterrows():
        report_buffer.write(f"| {row['run_no']} | {row['column']} | {row['Volume_ml']} | {row['Height_mAU']} | {row['Z-Score']} |\n")
else:
    report_buffer.write("No outliers detected (>2 ).\n")

report_buffer.write("\n## 3. Summary of Peak Volume_ml Consistency (Top 5 Profiles)\n")
report_buffer.write("| Normalized_Volume_ml | Frequency |\n")
report_buffer.write("|-----|-----|\n")
for profile in top_profiles_summary:
    report_buffer.write(f"| {profile['Normalized_Volume_ml']} | {profile['Frequency']} |\n")

# Save report to file
report_text = report_buffer.getvalue()
output_path = '/workspace/impurity_clustering_report.md'
with open(output_path, 'w') as f:
    f.write(report_text)

print(f"Report saved to {output_path}")

if __name__ == "__main__":
    main()

```

Listing 3: Code_3